Journal of Machine Learning Research 1 (2026) 1–5 Submitted 7/26; Published

# kodama-cpp: Cross-Validated Accuracy Maximization on CPU, CUDA, and Apple Metal

**Moussa Kassim**[1,2,*] MOUSSA.KASSIM@ICGEB.ORG
**Martin Ocharo**[1,2,*] MARTIN.OCHARO@ICGEB.ORG
**Dalia Ahmed**[1] DALIA.AHMED@ICGEB.ORG
**Dupe Ojo**[1] DUPE.OJO@ICGEB.ORG
**Alessia Vignoli**[3,4] VIGNOLI@CERM.UNIFI.IT
**Leonardo Tenori**[3,4,†] TENORI@CERM.UNIFI.IT
**Stefano Cacciatore**[1,2,†] STEFANO.CACCIATORE@ICGEB.ORG

[1] *Bioinformatics Unit, International Centre for Genetic Engineering and Biotechnology (ICGEB), Cape Town 7925, South Africa*
[2] *Department of Integrative Biomedical Sciences, Institute of Infectious Disease & Molecular Medicine (IDM), University of Cape Town, Cape Town 7925, South Africa*
[3] *Department of Chemistry "Ugo Schiff", University of Florence, Sesto Fiorentino, Italy*
[4] *Magnetic Resonance Center (CERM), University of Florence, Sesto Fiorentino, Italy*
[*] *Moussa Kassim and Martin Ocharo contributed equally.*
[†] *Leonardo Tenori and Stefano Cacciatore are co-corresponding authors.*

**Editor:** To be assigned

## Abstract

KODAMA discovers latent structure by maximizing held-out label predictability. This standalone C++17 implementation is shared by thin R and Python wrappers. Native multicore CPU, NVIDIA CUDA, and Apple Metal backends provide float32 KNN and SIMPLS–LDA without silent fallback. The library also introduces prediction-guided label evolution and exact-quota landmark projection. Tests distinguish numerical consistency and systems performance from external-label diagnostics.

**Keywords:** KODAMA, cross-validation, unsupervised learning, heterogeneous computing, manifold learning

## 1 Introduction

KODAMA treats labels as optimization variables: useful labels can be reproduced on held-out samples. An ensemble of optimized vectors then corrects neighborhood dissimilarities (Cacciatore et al., 2014, 2017). Unlike stability analysis or prediction strength, predictability is inside the search and no observed labels anchor it (Ben-Hur et al., 2002; Tibshirani and Walther, 2005).

Repeated cross-validation makes reusable state a systems concern. Our four contributions are **(1)** a typed C++ core shared by R and Python; **(2)** CPU, CUDA, and Metal under one float32 contract; **(3)** prediction-guided evolution with adaptive proposals, degeneracy guards, and error-scaled cooling; and **(4)** exact-quota landmark projection, formalizing the principle introduced by Abdel-Shafy et al. (2025).

## 2 KODAMA objective and algorithm

For data $X \in \mathbb{R}^{n \times p}$, labels $y$, folds $\Pi$, and classifier family $\mathcal{F}$, let

$$A(y; X, \mathcal{F}, \Pi) = \frac{1}{n} \sum_{i=1}^{n} \mathbf{1}\left[ y_i = \widehat{y}_i^{(-\Pi(i))} \right]. \tag{1}$$

KODAMA searches for high $A$ using either KNN or SIMPLS followed by latent-space LDA (de Jong, 1993). Folds stay fixed within each run, and supplied constraint groups move atomically. Each independent run draws exactly $L$ landmarks by population-proportional quotas from a coarse matrix partition; randomized systematic rounding fills the residual quota without replacement. A separate partition initializes the labels, so sampling strata are not optimization targets.

The classifiers share an evolution *scaffold*, not an identical state policy. Both use fixed folds, prediction-guided grouped proposals, the common transition matrix, one new CV pass per proposal, error-scaled cooling, and independent $M$ runs. Let $p_k$ be the class proportions, $D = 1 - \sum_k p_k^2$, $H = -\sum_k p_k \log p_k$, $K_{\text{eff}} = e^H$, and $R = \max\{0, \log[K / \max(1, K_{\text{eff}})]\}$. State selection uses

$$S_{\mathcal{F}}(y) = A(y)\sqrt{D} - \mathbf{1}_{\{\mathcal{F}=\text{PLS-LDA}\}}(1 - A(y)) \max\left\{ \frac{H}{\log n}, \frac{R}{1+R} \right\}. \tag{2}$$

Standard `KODAMA.matrix` applies transition-driven coarsening and the subtraction only to PLS–LDA; KNN uses the common diversity term alone. This reflects that LDA fits a mean per active class, whereas KNN has no analogous global class fit. Existing sensitivity results motivate, but do not isolate, this policy; no universal improvement is claimed.

The selected landmark vector is projected to all samples with the same classifier. If $a_{ij}$ is the fraction of projected runs agreeing on graph edge $(i, j)$, KODAMA returns $d'_{ij} = (1 + d_{ij})/a_{ij}^2$ and removes zero-agreement edges. Exact proposal, temperature, quota, and coarsening formulas are given in the supplement.

**Algorithm 1: KODAMA ensemble**
**Input:** $X, \mathcal{F}, M, T, L, K_0, \Pi, s$. **Output:** $G, C, A_{\text{best}}, Z_U, Z_T$.

$X_{32} \leftarrow$ Float32($X$); $(G_0, Z_U, Z_T) \leftarrow$ KODAMA.Graph($X_{32}$) once
**for** $r = 1, \ldots, M$ **do**
  $I_r \leftarrow$ ExactQuotaSample(Partition($X, K_0$), $L, s + r$)
  $y \leftarrow$ InitialPartition($X_{I_r}, K_0$); $(A, \widehat{y}) \leftarrow \text{CV}(\mathcal{F}, X_{I_r}, y, \Pi_r)$
  **for** $t = 1, \ldots, T$ **do**
    $y' \leftarrow$ GuidedProposal($y, \widehat{y}, t, T$); if $\mathcal{F} =$ PLS-LDA, apply transition coarsening
    $(A', \widehat{y}') \leftarrow \text{CV}(\mathcal{F}, X_{I_r}, y', \Pi_r)$; update by $S_{\mathcal{F}}$ and cooling
  **end for**
  $C_{\cdot r} \leftarrow$ Project($\mathcal{F}, X_{I_r}, y_{\text{best}}, X$)
**end for**
$G \leftarrow$ AgreementCorrectionInPlace($G_0, C$); **return** $G, C, A_{\text{best}}, Z_U, Z_T$

Figure 1: Compact pseudocode. The scaffold is common; the indicated coarsening and score term are PLS–LDA-specific.

## 3 Standalone heterogeneous implementation

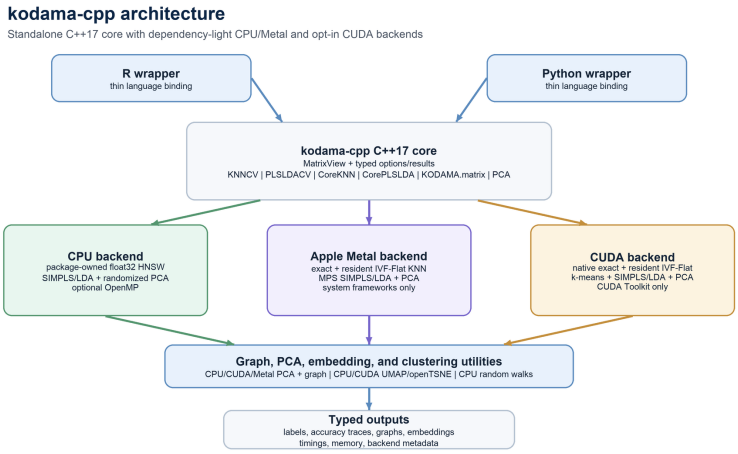


Figure 2: The C++17 core owns algorithm state; R and Python are thin adapters. For a fixed classifier, backends share folds, policy, component semantics, and typed results.

The typed C++ API owns folds, proposals, classifiers, landmark projection, graph correction, and metadata; wrappers only convert host objects and results. CPU supplies parallel HNSW construction over contiguous float32 storage (Malkov and Yashunin, 2020), with a deterministic connected base seed before lock-protected concurrent insertion and querying, plus SIMPLS–LDA, PCA, graph operations, and embeddings. CUDA supplies exact/IVF KNN, batched k-means, label-aware SIMPLS–LDA, PCA, and embeddings. Metal supplies exact/IVF KNN, k-means, MPS-assisted SIMPLS–LDA, and PCA. Accelerator IVF construction and persistent indices retain training data and index state on-device. During KODAMA, one resident full-data graph and persistent worker-lane buffers retain fold layouts, labels, projections, voting state, and PLS–LDA workspaces across proposal cycles; contents that depend on a new landmark sample are refreshed in place once per $M$ run. Unsupported backends raise an error rather than silently falling back.

PLS–LDA forms class cross-products without dense one-hot responses, compacts active labels, reuses fold workspaces, and evaluates the requested feasible component count without model selection. `KODAMA.graph` prepares one full-data neighbor graph and separately scaled backend-matched UMAP/openTSNE PCA starts. Its result owns only graph arrays, starts, and metadata: it never retains the raw matrix. The wrappers reserve `data` for raw features and `graph` for a prepared object or bare paired $n \times k$ index/distance graph; either can be supplied alone or both can be supplied together, and raw-only input invokes graph preparation internally. Graph-only execution uses disclosed self-tuning normalized-Laplacian geometry, including the PLS–LDA spectral surrogate, whereas supplying raw features retains ordinary data-input PLS–LDA. Fixed-graph KNN reuses supplied neighborhoods and is distinguished from data-native KNN rather than asserted to reproduce its stochastic trajectory. Final compaction and agreement correction mutate the single graph owner in place. The API

also exposes both CV kernels, both core optimizers, PCA, and float32 UMAP/openTSNE (McInnes et al., 2018; van der Maaten and Hinton, 2008). Explicit visualization starts take precedence, then backend-matched stored starts, then raw-data recomputation and graph-only fallbacks; provenance is reported. Derivations, safeguards, pseudocode, evidence, and API contracts are in the supplement.

## 4 Validation, availability, and scope

Table 1: Matched runtime and held-out accuracy. Times are seconds; KODAMA reports median raw CV accuracy for $M = T = 100$.

| Scope | Data | Accel. | CPU | Device | Speedup | Accuracy |
|---|---|---|---|---|---|---|
| KNNCV | MNIST | CUDA | 76.769 | 4.753 | 16.2 | .974/.973 |
| PLSLDACV | MetRef | CUDA | 1.294 | 0.308 | 4.20 | .992/.993 |
| KODAMA PLS–LDA | MetRef | CUDA | 341.648 | 270.408 | 1.26 | .9924/.9924 |
| KNNCV | MetRef | Metal | 11.145 | 0.026 | 425.0 | .827/.827 |
| PLSLDACV | MetRef | Metal | 0.790 | 0.202 | 3.90 | .991/.990 |

Table 1 separates kernels from the complete KODAMA call. CUDA and Metal preserved closely matched held-out accuracy, while end-to-end acceleration was smaller because repeated orchestration and graph work remain. Finite-precision ties may select different stochastic trajectories, so numerical consistency is assessed by contracts and paired metrics rather than bitwise label identity. Extended runtime, memory, landmark, $M/T$, visualization, and prior-version comparisons are reported in the supplement.

The fixed-$M = 100$ sensitivity study also shows why classifier-independent scoring is not claimed: at $T = 100$, median active-class counts were 9 versus 24 on MetRef and 32 versus 71 on USPS for KNN versus PLS–LDA. These observations establish different fragmentation regimes, but because they are not penalty-on/off ablations they do not establish the causal benefit of the PLS–LDA term.

CTest and wrapper suites exercise float32 numerics, strict backend identity, graph and PCA contracts, backend-matched visualization initialization, landmark counts, and the candidate-`0.1.0` API. A controlled CPU openTSNE comparison against the pinned fastEmbedR source was coordinate-identical. GitHub Actions covers Linux/macOS CPU, native Metal, wrappers, coverage, and API documentation; CUDA is validated on a dedicated NVIDIA host. The audited CPU snapshot reached 63.58% line and 57.03% branch coverage.

The MIT-licensed core is available at `https://github.com/tkcaccia/kodama-cpp`; the R wrapper is at `https://github.com/tkcaccia/kodama-r`, and tested Python wrapper source accompanies the core pending its standalone repository. Repeated CV remains the dominant cost. Graph and classifier workspaces are device resident, but landmark selection, proposal decisions, and compact result collection remain host orchestrated; approximate-neighbor ties can alter trajectories, and CUDA graph construction currently supports $k \leq 256$.

## References

Ebtesam A. Abdel-Shafy, Moussa Kassim, Alessia Vignoli, Farag Mamdouh, Svitlana Tyekucheva, Dalia Ahmed, Dupe Ojo, Brendon Price, Fabio Socciarelli, Nancy Paola Duarte-Delgado, Martin Ocharo, Chiamaka Jessica Okeke, Luca Triboli, David A. MacIntyre, Massimo Loda, Devanand Sarkar, Dinesh Gupta, Silvano Piazza, Luiz Fernando Zerbini, Leonardo Tenori, and Stefano Cacciatore. Kodama enables self-guided weakly supervised learning in spatial transcriptomics. *bioRxiv*, 2025. doi: 10.1101/2025.05.28.656544. Preprint.

Asa Ben-Hur, Andre Elisseeff, and Isabelle Guyon. A stability based method for discovering structure in clustered data. In *Pacific Symposium on Biocomputing*, pages 6–17, 2002.

Stefano Cacciatore, Claudio Luchinat, and Leonardo Tenori. Knowledge discovery by accuracy maximization. *Proceedings of the National Academy of Sciences*, 111(14):5117–5122, 2014.

Stefano Cacciatore, Leonardo Tenori, Claudio Luchinat, Phillip R. Bennett, and David A. MacIntyre. Kodama: an r package for knowledge discovery and data mining. *Bioinformatics*, 33(4):621–623, 2017. doi: 10.1093/bioinformatics/btw705.

Sijmen de Jong. Simpls: an alternative approach to partial least squares regression. *Chemometrics and Intelligent Laboratory Systems*, 18(3):251–263, 1993.

Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2020.

Leland McInnes, John Healy, and James Melville. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv:1802.03426*, 2018.

Robert Tibshirani and Guenther Walther. Cluster validation by prediction strength. *Journal of Computational and Graphical Statistics*, 14(3):511–528, 2005.

Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of Machine Learning Research*, 9:2579–2605, 2008.